

Distributed training client configuration

This has PyTorch + boto3-style access, streaming shards directly from object storage:

```
import os

os.environ["AWS_ACCESS_KEY_ID"] = "training-service-account"
os.environ["AWS_SECRET_ACCESS_KEY"] = "REDACTED"
os.environ["AWS_ENDPOINT_URL"] = "http://minio.internal:9000"

# WebDataset-style streaming from object storage into the
# training loop
import webdataset as wds

url = "pipe:mc cat local/training-datasets/imagenet-1k/train/
shard-{00000..01023}.tar"
dataset = wds.WebDataset(url).decode("pil").to_tuple("jpg",
"cls")
```

Recommended tuning for training throughput

Optimise petabyte-scale training data storage on MinIO or Ceph by deploying erasure coding instead of full replication to maximise capacity efficiency for read-heavy workloads. You can prevent data-loading bottlenecks by co-locating storage nodes with GPU nodes on a high-bandwidth (25/100 GbE) network fabric. Minimise object storage request overhead by sharding datasets into 100MB-1GB .tar files following the WebDataset convention. Finally, implement client-side NVMe caching to handle frequently reused epochs when memory capacity is exceeded.

Case study

We look at a mid-sized AI research lab, with a target of 3PB of training data, coming from an existing capacity of 200TB. Previously, training datasets were accommodated by a single large NFS server. With the passage of time, lab data began to accumulate in versions (multiples of checkpoints and copies of the augmented processed and unprocessed corpora) to the point that it reached 200TB. From that point onwards, the lab began to experience degraded NFS performance, particularly during large-scale, multi-GPU, distributed training. Data loader workers began to spend more time waiting for I/O than on GPU compute.

So this is what they did:

- Deployed a 5-node Ceph cluster (60 OSDs total) as the durable long-term store for all raw and processed datasets, using erasure coding ($k=6$, $m=3$) to balance durability against the 66% storage overhead of 3x replication.
- Deployed 3 MinIO nodes with local NVMe drives, positioned as a fast S3-compatible cache/gateway layer directly on the GPU training cluster's network segment.
- Used Ceph's RGW multisite replication to periodically

sync 'hot' datasets (the current training epoch's shards) into the MinIO layer, while cold/archival data stayed exclusively in Ceph.

- Migrated all training scripts to use the S3 API (via boto3/s3fs) pointed at the MinIO endpoint, eliminating NFS entirely from the training path.

The outcome was:

- Dataloader wait time during training dropped substantially once data was served from MinIO's NVMe-backed nodes on the same network segment as the GPUs, removing the previous I/O bottleneck.
- Storage costs stayed roughly flat year-over-year despite data volume growing more than 10x, since erasure-coded Ceph on commodity hardware scaled far more cheaply than adding proprietary NAS capacity.
- The self-healing behaviour of both Ceph and MinIO meant several individual disk failures over the following year required zero manual data recovery — the systems rebuilt redundancy automatically.
- The team gained the ability to version and roll back entire dataset snapshots (via MinIO bucket versioning) after a mislabelled data ingestion incident, something the prior NFS setup had no equivalent for.

This pattern — Ceph as the durable multi-petabyte backend and MinIO as a fast S3-native layer close to compute — has become a common reference architecture for AI infrastructure teams operating at this scale without relying on public cloud storage.

At the petabyte scale, Ceph and MinIO address different aspects of the same problem. Ceph's storage enables ease-of-use, multi-protocol, seamless scaling to large clusters, and self-healing storage, while MinIO provides S3-native, high-speed storage to support ML training with minimal latency. When Ceph is employed as the long-term storage solution and MinIO as a fast S3 gateway collocated with processing, durability and cost efficiencies are realised without loss of the performance necessary to support modern training pipelines.

The process to fully scale out each system may be slow. We can start with a small Ceph cluster and a single MinIO instance to confirm our setup is correct. We then ensure everything is healthy and scale out nodes. We keep expanding as the data grows. As the software is open source, it lets us keep that same layout with some additional hardware as the capacity of our research lab grows from 200TB to multiple petabytes. **END** 🐧

 By: Dr R. Anand and Dr S.Palaniappan Sambandam

Dr R. Anand is a professor in the Department of Computer Science and Business Systems, Easwari Engineering College.

Dr S. Palaniappan Sambandam heads the Department of Computer Science and Business Systems, Easwari Engineering College.